AI Code Navigator - Week 3 and Week 4
Progress Report Project: AI Code Navigator
Name: Naman Jhala

Problem: Search Was Returning Whole Files Instead of Exact Functions

What Was the Problem?

When someone searched the codebase with a question like *"how does authentication work?"*, the system would return the entire auth.js file - even if only one function inside that file was relevant. This caused two issues:

- **Too much noise.** The AI got a 200-line file when it only needed a 15-line function. Extra code confuses the answer.
- **Low search precision.** Every function in the same file shared one single embedding (one "fingerprint"). So searching for *"how does login work?"* and *"how does logout work?"* might return the exact same file with the same confidence score - even if those two functions are completely different.
- **Wasted tokens.** Passing a whole file to GPT-4o-mini when only one function was needed used more tokens than necessary.

How Did We Fix It? (AST Chunking)

We added a step called **AST chunking** - AST stands for Abstract Syntax Tree. It's a way for the computer to "read" a JavaScript or TypeScript file and understand its structure, not just its raw text.

Before storing a file, we now:

1. **Parse the file** using a tool called **Babel** (already installed in the project). Babel reads the JS/TS code and identifies every function and class inside it.
2. **Split it into chunks.** Each function becomes its own chunk. Each class becomes its own chunk.
3. **Store each chunk separately** in MongoDB - with its own AI summary, its own embedding (vector fingerprint), and its own name (e.g., authenticate, hashCode, UserController).

4. **For non-JS files** (like .py, .json, .md), nothing changes - we still store the whole file as one document.

Now when someone searches *"how does authentication work?"*, the system finds the specific authenticate function - not the whole file.

What Files We Changed

1. services/astService.js *(new file)*

This is the brain of the chunking logic. It takes a file's code and name, uses Babel to parse it, and returns a list of chunks like:

{ chunkName: "authenticate", chunkType: "function", code: "function authenticate() { ... }" }

{ chunkName: "hashPassword", chunkType: "function", code: "const hashPassword = ..." }

{ chunkName: "UserController", chunkType: "class", code: "class UserController { ... }" }

It handles four types of JS/TS declarations:

- Regular functions: function foo() {}
- Arrow/expression functions: const foo = () => {}
- Classes: class Foo {}
- Exported defaults: export default function foo() {}

If the file is not JS/TS, or if Babel can't parse it, the function returns null so the rest of the system falls back to whole-file processing.

---

2. models/code.js *(updated)*

Added three new fields to the MongoDB document schema:

- fileHash - a SHA-256 hash of the **whole file** (used to detect duplicates - if the file hasn't changed, we skip all processing)
- chunkName - the name of the function or class
  (e.g., "authenticate", "UserController")
- chunkType - one of "function", "class", or "file" (for non-JS files stored as a whole)

---

3. controllers/codecontroller.js *(updated)*

This is where all upload and search logic lives. We made three changes:

Added a shared helper function processAndSave() - all three upload paths (single snippet, file upload, GitHub repo) now use this one function instead of repeating the same logic. It:

- Checks fileHash to skip duplicates
- Calls chunkByAST() for JS/TS files
- Saves each chunk (or whole file) to MongoDB

Updated all three upload paths (processCode, uploadCode, uploadRepo) to use the new processAndSave() helper.

Updated searchCode to return chunkName and chunkType in the response, so the caller can see exactly which function matched - not just which file.

How We Tested It

To verify the chunking works, you can test it with a sample JS file containing multiple functions:

Test 1 - Upload a JS file with multiple functions:

POST /api/upload

Files: [auth.js containing: hashPassword(), authenticate(), generateToken()]

Expected: MongoDB now has 3 separate documents, one per function, each with its own summary and embedding.

Test 2 -Search for a specific function:

POST /api/search

Body: { "query": "how does token generation work?" }

Expected response now includes:

{

"answer": "The generateToken function creates a JWT signed with...",

"relevantChunks": [

{ "fileName": "auth.js", "chunkName": "generateToken", "chunkType": "function" }

]

}

Test 3 - Upload a Python file:

POST /api/upload

Files: [utils.py]

Expected: stored as one whole document, chunkType: "file" no AST splitting (Python not supported by Babel).

Test 4 -Duplicate detection: Upload the same JS file twice. Second upload should be skipped because fileHash already exists in the database.

Result

	Before (Week 12)	**After (Week 13)**
What gets stored	1 document per file	1 document per function/class in JS/TS
Search result	Returns the whole file	Returns the exact function that matched
Search response	relevantFiles: ["auth.js"]	relevantChunks: [{ fileName: "auth.js", chunkName: "authenticate", chunkType: "function" }]
Non-JS files	Whole file	Unchanged — still whole file
Duplicate check	Per-file SHA-256 hash	Per-file fileHash (same logic, cleaner field name)
New files added	0	1 (services/astService.js)
Files modified	0	2 (models/code.js, controllers/codecontroller.js)
